

Coding Interview Problems — Explained for New Joiners

Six classic problems, explained from first principles with the intuition first, then working (tested) Python.

Contents

1. [Combination Sum](#)
 2. [Trapping Rain Water](#)
 3. [Lowest Common Ancestor of a Binary Tree](#)
 4. [Maximum Subarray — Kadane's Algorithm](#)
 5. [Longest Substring Without Repeating Characters](#)
 6. [Minimum Window Substring](#)
-

1. Combination Sum

The problem, in plain English

You're given a list of **distinct positive numbers** (say, coin denominations) and a **target** amount. You want every possible way to add up numbers from the list so they total exactly the target — and you're **allowed to reuse the same number as many times as you like**.

Two combinations count as "the same" if they use the same numbers the same number of times, regardless of order — so `[2, 3]` and `[3, 2]` are not counted twice.

Example: `candidates = [2, 3, 6, 7]`, `target = 7`
Valid combinations: `[2, 2, 3]` ($2+2+3=7$) and `[7]` ($7=7$). Nothing else works.

How to think about it

Imagine you're building a combination one number at a time, left to right. At each step you have two kinds of choices: "add this number again" or "move on to the next candidate and stop using this one." If you only ever move *forward* through the candidate list (never back to a number you've already skipped), you can never produce the same combination twice — because the numbers in every combination you build will always come out in non-decreasing order.

This is a classic **backtracking** (try something, undo it, try the next thing) pattern:

- Keep a running "remaining target" (starts at the target, shrinks as you pick numbers).
- If remaining hits exactly `0`, you've found a valid combination — save it.
- If remaining goes negative, this path is a dead end — back out immediately.

- At each candidate, you either use it again (recurse without advancing the index, since reuse is allowed) or skip it for good (recurse from the next index).

Sorting the candidates first lets you stop early the moment a candidate is bigger than what's left — no smaller number will appear later to fix that.

Python

```
def combination_sum(candidates, target):
    result = []
    candidates.sort()

    def backtrack(start, remaining, path):
        if remaining == 0:
            result.append(path[:])
        # found a valid combination
        return
        for i in range(start,
            len(candidates)):
            if candidates[i] > remaining:
                break
        # sorted list -> nothing further can work
        either
            path.append(candidates[i])
            backtrack(i, remaining -
            candidates[i], path)  # 'i' not 'i+1':
            reuse allowed
            path.pop()
        # undo the choice, try the next one
```

```
        backtrack(0, target, [])
    return result

print(combination_sum([2, 3, 6, 7], 7))    #
[[2, 2, 3], [7]]
print(combination_sum([2, 3, 5], 8))      #
[[2, 2, 2, 2], [2, 3, 3], [3, 5]]
```

Complexity: exponential in the worst case (it's an exhaustive search), but the "sort + break early" trick prunes a large fraction of dead-end paths in practice. Space is $O(\text{target} / \text{smallest candidate})$ for the recursion depth.

2. Trapping Rain Water

The problem, in plain English

You're given a row of bars of different heights (think of a bar chart, or a cross-section of a city skyline). It rains. Water pools in the "dips" between taller bars and runs off the edges. How much total water ends up sitting on top of the bars?

Example: heights `[0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]` traps 6 units of water.

How to think about it

Water sitting *above* any single bar is trapped by whichever is shorter: the tallest bar somewhere to its left, or the tallest bar somewhere to its right. Water always finds the lowest escape route, so the **shorter of the two "walls"** is what actually determines how high the water can pile up at that spot.

For a given position `i`:

```
water_at(i) = max(0,  
    min(max_height_to_the_left_of_i,  
        max_height_to_the_right_of_i) - height[i])
```

Straightforward approach: precompute, for every index, the tallest bar to its left and the tallest bar to its right (two passes, `O(n)` time, `O(n)` extra memory), then apply the formula above at every index.

Smarter approach — two pointers, `O(1)` extra memory: walk from both ends toward the middle at once, tracking `left_max` and `right_max` as you go. Here's the key trick that makes this safe: if `left_max ≤ right_max` right now, then the water level at the current *left* pointer is **already fully determined** — it's capped by `left_max`, because no matter what's still unexplored on the right side, the right side's max is already at least as big, so it can never become the limiting wall. That means you can safely compute the water at the left pointer and move it inward.

Whichever side currently has the *smaller* max is always the side that's safe to resolve next.

Python

```
def trap(height):
    if not height:
        return 0

    left, right = 0, len(height) - 1
    left_max, right_max = height[left],
height[right]
    water = 0

    while left < right:
        if left_max <= right_max:
            left += 1
            left_max = max(left_max,
height[left])
            water += left_max -
height[left]      # always >= 0 by
construction
        else:
            right -= 1
            right_max = max(right_max,
height[right])
            water += right_max -
height[right]

    return water
```

```
print(trap([0,1,0,2,1,0,1,3,2,1,2,1])) # 6
print(trap([4,2,0,3,2,5])) # 9
```

Complexity: $O(n)$ time, $O(1)$ extra space — each pointer moves inward once, so the whole array is scanned only once total.

3. Lowest Common Ancestor of a Binary Tree

The problem, in plain English

Given a general binary tree (no ordering guarantees — this is *not* a binary search tree) and two nodes `p` and `q` somewhere in it, find their **lowest common ancestor (LCA)**: the deepest node that has *both* `p` and `q` as descendants (a node counts as a descendant of itself). Think of an org chart — the LCA is the closest shared manager of two employees.

How to think about it

Since there's no ordering to exploit (you can't tell whether to go left or right just by comparing values, the way you could in a BST), the natural approach is: **search the whole tree, and let information bubble back up from the leaves toward the root.**

At any node, ask recursively: "does `p` or `q` (or both) live somewhere in my left subtree? my right subtree?"

- If the current node **is** `p` or `q`, report that immediately (it's found itself).
- If **both** the left and right recursive calls report "found something," that means `p` and `q` live in *different* subtrees of the current node — which makes the current node their meeting point, i.e., the LCA.
- If only **one side** reports something, just pass that result further up — it means both nodes are (so far) on the same side, or only one of the two nodes has been located yet.

The elegance here is that the same recursive call does double duty: it both *searches* for `p / q` and *identifies* the LCA the moment their paths diverge.

Python

```
class TreeNode:
    def __init__(self, val=0, left=None,
right=None):
        self.val = val
        self.left = left
        self.right = right

def lowest_common_ancestor(root, p, q):
    if root is None or root == p or root ==
q:
```

```

        return root

    left =
lowest_common_ancestor(root.left, p, q)
    right =
lowest_common_ancestor(root.right, p, q)

    if left and right:
        return root          # p and q
found on different sides -> this is the LCA
        return left if left else right    #
otherwise pass up whichever side found
something

# Tree:      3
#           / \
#          5  1
#         / \ / \
#        6  2 0  8
#       / \
#      7  4
n7, n4 = TreeNode(7), TreeNode(4)
n2 = TreeNode(2, n7, n4)
n6 = TreeNode(6)
n5 = TreeNode(5, n6, n2)
n0, n8 = TreeNode(0), TreeNode(8)
n1 = TreeNode(1, n0, n8)
root = TreeNode(3, n5, n1)

print(lowest_common_ancestor(root, n5,
n1).val) # 3
print(lowest_common_ancestor(root, n5,
n4).val) # 5

```

Complexity: $O(n)$ time — in the worst case every node has to be visited once, since nothing about a general binary tree tells you where p / q are without looking. $O(h)$ space for the recursion stack, where h is the tree's height.

Side note: if the tree were a binary search tree, you could do this in $O(h)$ time without visiting every node — just compare $p.val$ and $q.val$ against the current node's value to decide whether to go left, right, or stop, since BST ordering tells you exactly which subtree each node lives in.

4. Maximum Subarray — Kadane's Algorithm

This is the same problem as "find the contiguous subarray with the largest sum" — Kadane's algorithm *is* the standard solution to it.

The problem, in plain English

Given an array of numbers (positive, negative, or zero), find the **contiguous** run of numbers (at least one number) whose sum is the largest possible.

Example: $[-2, 1, -3, 4, -1, 2, 1, -5, 4] \rightarrow$ the best run is $[4, -1, 2, 1]$, summing to 6.

How to think about it

Picture the numbers as daily profit/loss on a stock, and you're choosing the best possible *contiguous* stretch of days to have held it. At each day, you're really only asking one question: **"is it better to keep extending yesterday's streak, or to start fresh from today?"**

That's exactly Kadane's insight: keep a running sum representing "the best subarray sum that ends *exactly* here." At each new number, either extend the previous running sum by adding the new number, or abandon everything before it and start over at the new number alone — whichever gives a bigger result. You'd only ever want to restart if the running sum so far had gone negative, since a negative prefix can only drag down anything that follows it. Track the best value seen at any point along the way — that's your answer.

Python

```
def max_subarray(nums):
    current_sum = best_sum = nums[0]
    for num in nums[1:]:
        current_sum = max(num, current_sum
+ num)    # extend, or restart here
        best_sum = max(best_sum,
current_sum)
    return best_sum
```

```
print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4])) # 6
```

Complexity: $O(n)$ time, $O(1)$ space — a single pass, two running variables, no extra memory.

5. Longest Substring Without Repeating Characters

The problem, in plain English

Given a string, find the length of the longest **contiguous** stretch (substring) that has no repeated characters.

Example: "abcabcbb" → the answer is 3 ("abc").

How to think about it

Use a **sliding window**: two pointers, `left` and `right`, marking the current "no duplicates yet" window. Slide `right` forward one character at a time, growing the window. The moment the incoming character is already inside the current window, you have a duplicate — so you need to shrink from the `left` side until that duplicate is gone.

The efficient trick: instead of shrinking one character at a time and re-checking, keep a dictionary of **the last**

index each character was seen at. When you hit a repeat, you can jump `left` directly to just past the previous occurrence — no need to inch forward one step at a time. (The `>= left` check matters: it makes sure you don't jump backward based on a character that appeared *before* the current window even started.)

Python

```
def length_of_longest_substring(s):
    last_seen = {}
    left = 0
    longest = 0

    for right, char in enumerate(s):
        if char in last_seen and
last_seen[char] >= left:
            left = last_seen[char] + 1
        # jump past the earlier occurrence
        last_seen[char] = right
        longest = max(longest, right - left
+ 1)

    return longest

print(length_of_longest_substring("abcabcbb
")) # 3
print(length_of_longest_substring("pwwkew")
) # 3 ("wke")
```

Complexity: $O(n)$ time — each character is looked at a constant number of times. $O(\min(n, \text{alphabet size}))$ space for the dictionary.

6. Minimum Window Substring

The problem, in plain English

Given two strings S and T , find the **shortest contiguous substring of S** that contains **every character of T** , including repeats — if T has two 'a' s, your window needs at least two 'a' s too. If no such window exists, return an empty string.

Example: $S = \text{"ADOBECODEBANC"} , T = \text{"ABC"} \rightarrow$ the answer is "BANC" .

How to think about it

Think of T as a shopping list with required quantities (2 apples, 1 banana, ...), and S as one long aisle you walk down. You want the shortest continuous stretch of the aisle that satisfies the entire list.

This is another **sliding window**, but this time you track counts, not just presence:

- Build a "required" count of every character in T .
- Expand the window by moving **right** forward, updating how many of each needed character you

currently have.

- Keep a counter, `formed`, for how many *distinct* required characters currently have **enough** copies in the window (matching or exceeding what `T` needs).
- The moment `formed` equals the total number of distinct characters `T` needs, your window is *valid* – it satisfies the whole shopping list. At that point, try to shrink from the `left` as much as possible while it's still valid, recording the smallest valid window you've seen so far each time before it stops being valid.
- Keep repeating: grow with `right`, shrink with `left`, and remember the best window found.

Every character in `S` is only ever touched by `right` once (when entering the window) and by `left` at most once (when leaving it), which is what keeps this linear time despite the nested-looking loops.

Python

```
from collections import Counter

def min_window(s, t):
    if not s or not t:
        return ""

    need = Counter(t)
    required = len(need)          # number
```

of **distinct** characters that must be satisfied

```
    window_counts = {}
    formed = 0                # how
many distinct required chars are currently
satisfied
    left = 0
    best_len, best_left, best_right =
float("inf"), 0, 0

    for right, char in enumerate(s):
        window_counts[char] =
window_counts.get(char, 0) + 1
        if char in need and
window_counts[char] == need[char]:
            formed += 1

    while formed == required:
# window is currently valid -> try to
shrink it
        if right - left + 1 < best_len:
            best_len = right - left + 1
            best_left, best_right =
left, right

        left_char = s[left]
        window_counts[left_char] -= 1
        if left_char in need and
window_counts[left_char] < need[left_char]:
            formed -= 1
# shrinking broke validity, stop shrinking
next loop
        left += 1
```

```
        return "" if best_len == float("inf")
    else s[best_left:best_right + 1]

print(min_window("ADOBECODEBANC", "ABC"))
# "BANC"
print(min_window("a", "aa"))
# "" (S doesn't have two 'a's)
```

Complexity: $O(|S| + |T|)$ time — building the `need` counter is $O(|T|)$, and each character of `S` is processed by `right` and `left` a constant number of times. $O(|S| + |T|)$ space for the counters in the worst case.

All six snippets above were executed and checked against known expected outputs before inclusion in this document.